SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**THIRUMALAPUDI PRANAVI**	**Reg.No.:**	**192411222**
Department:	**BE-CSE**	**Date:**	**29-08-2026**

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 µs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I THIRUMALAPUDI PRANAVI (192411222)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:T.Pranavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Construct an I/O Communication Mechanism for a Real-Time Embedded System

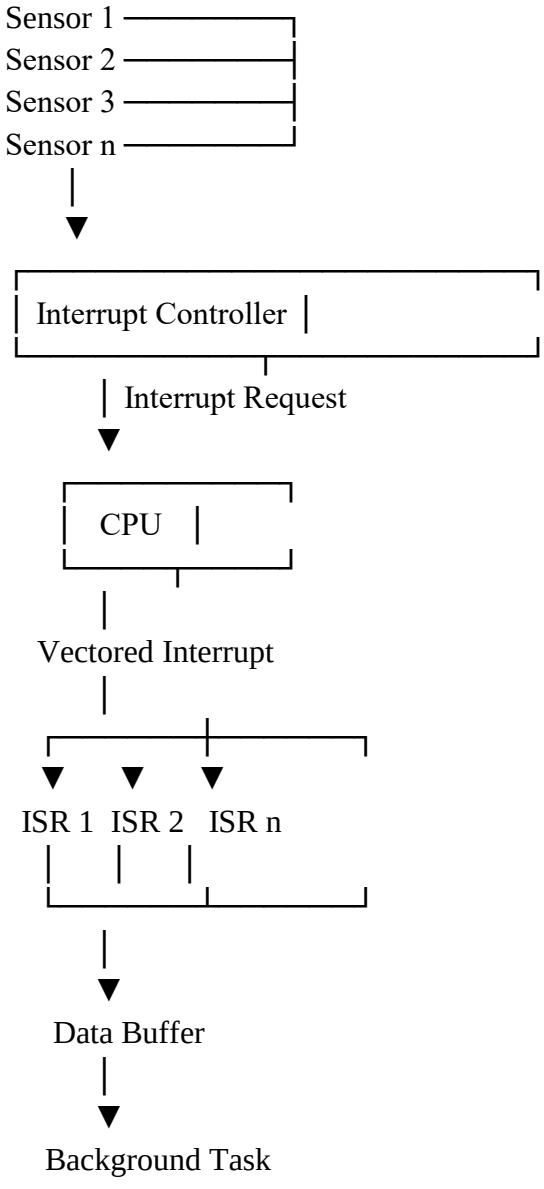
Given Constraints

- **CPU utilization must be below 15%**
- **Multiple sensors generate interrupts every 1 ms**
- **Use vectored interrupt handling**
- **Select between memory-mapped I/O and I/O-mapped I/O for optimal latency**

Aim

To design an efficient I/O communication mechanism for a real-time embedded system that handles multiple sensor interrupts with low latency and CPU utilization below 15%.

Block Diagram



Selected I/O Method

Memory-Mapped I/O

Memory-mapped I/O is selected for this system.

Reason

- Device registers are accessed like memory locations.
- The CPU can use normal load and store instructions.
- It provides simple and fast access to sensor registers.
- It is suitable for a real-time embedded system requiring low latency.

Working Procedure

Step 1: Sensor Generates an Interrupt

Each sensor generates an interrupt every **1 ms** when new data is available.

Sensor → Interrupt Request

Step 2: Interrupt Controller Identifies the Sensor

The interrupt controller receives requests from multiple sensors and determines which interrupt has occurred.

Sensors → Interrupt Controller → CPU

Step 3: Vectored Interrupt Selects the ISR

Each sensor is assigned a unique interrupt vector.

Sensor	Interrupt Vector	ISR
Sensor 1	Vector 1	ISR 1
Sensor 2	Vector 2	ISR 2
Sensor 3	Vector 3	ISR 3
Sensor n	Vector n	ISR n

The CPU directly jumps to the required Interrupt Service Routine.

Interrupt from Sensor 2

↓

Interrupt Vector 2

↓

Execute ISR 2

Step 4: ISR Reads the Sensor Data

The ISR quickly reads the sensor's data through memory-mapped I/O.

Sensor Register

↓

Memory-Mapped Address

↓

ISR

Step 5: Store Data in a Buffer

The sensor data is placed into a memory buffer.

Sensor → ISR → Buffer

The ISR should not perform lengthy calculations.

Step 6: Return from the Interrupt

After storing the data, the ISR immediately returns.

ISR Complete

↓

Return to Previous Program

Step 7: Background Processing

The main program or a background task processes the buffered sensor data later.

Data Buffer → Background Processing

How CPU Utilization Is Kept Below 15%

The following techniques are used:

1. Keep the ISR very short.
2. Use vectored interrupts to directly locate the ISR.
3. Avoid continuous polling.
4. Store incoming data in buffers.
5. Perform complex processing in the background.
6. Use priority-based interrupt handling if some sensors are critical.

Why Vectored Interrupts?

Without vectored interrupts:

Interrupt

↓

Check Device 1?

↓

Check Device 2?

↓

Check Device 3?

This increases latency.

With vectored interrupts:

Interrupt

↓

Vector Number

↓

Directly Execute Required ISR

This reduces interrupt response time.

Final Design

Multiple Sensors
↓
Interrupt Controller
↓
Vectored Interrupt
↓
Short ISR
↓
Memory-Mapped I/O
↓
Data Buffer
↓
Background Processing

Result

Thus, the real-time embedded I/O communication mechanism uses memory-mapped I/O and vectored interrupt handling. Each sensor interrupt is directed immediately to its corresponding ISR, sensor data is stored in a buffer, and complex processing is deferred to background tasks. This provides low latency while maintaining CPU utilization below 15%.

2. Design an I/O Subsystem for High-Speed Storage

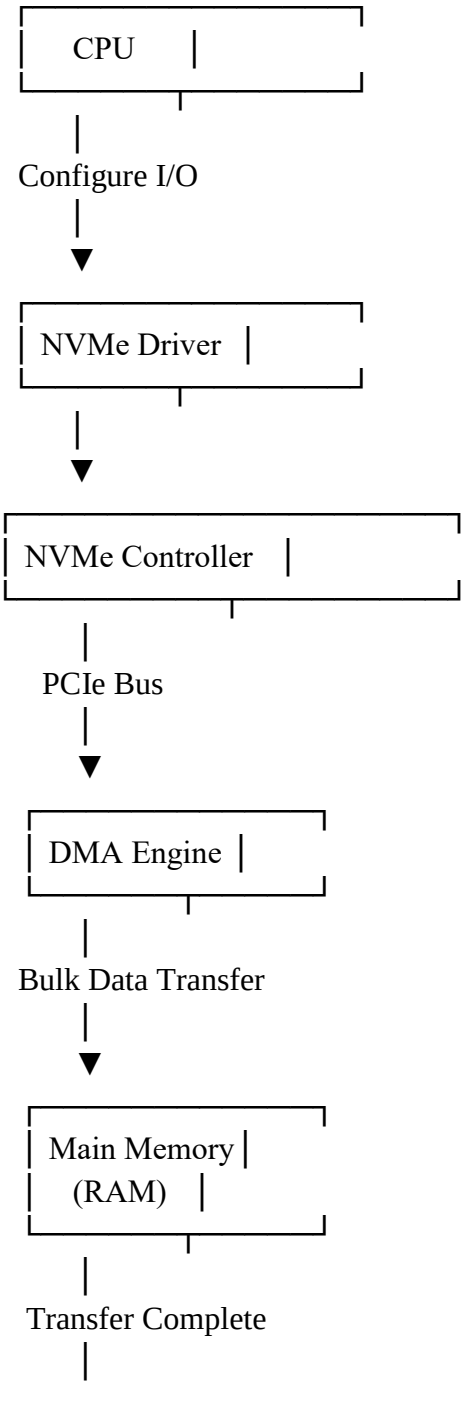
Given Constraints

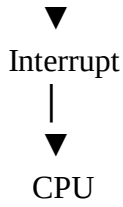
- **High-speed storage requires throughput greater than 3 GB/s**
- **Must use NVMe over PCIe**
- **CPU should not handle bulk data transfer**
- **Integrate a DMA controller with interrupt notification**

Aim

To design a high-speed I/O subsystem using **NVMe over PCIe and DMA** to achieve more than 3 GB/s throughput with minimal CPU involvement.

Block Diagram





Components Used

Component	Function
CPU	Initializes and controls the transfer
NVMe SSD	Stores high-speed data
PCIe	Provides a high-speed communication path
NVMe Controller	Manages NVMe commands
DMA Controller	Transfers bulk data directly
Main Memory	Stores transferred data
Interrupt Controller	Notifies CPU after completion

Working Procedure

Step 1: CPU Receives an I/O Request

The CPU receives a request to read or write data.
For example:
Application
↓
Operating System
↓
NVMe Driver
The CPU only initiates the transfer and does not move every byte of data.

Step 2: NVMe Command Is Created

The NVMe driver creates a command containing:

- Read or write operation
- Storage address
- Memory address
- Size of data

CPU → NVMe Command Queue

Step 3: Command Travels Through PCIe

The NVMe command is sent to the NVMe SSD through the PCIe interface.
CPU
▼
NVMe Controller
▼
PCIe Bus

NVMe SSD

PCIe is selected because it provides the high bandwidth required for throughput greater than **3 GB/s**.

Step 4: DMA Is Configured

The CPU configures the DMA controller with:

Source Address → NVMe SSD

Destination Address → Main Memory

Transfer Size → Required Data Size

Operation → Read/Write

After configuration, the CPU is free to perform other tasks.

Step 5: Bulk Data Transfer Takes Place

The DMA engine transfers data directly between storage and main memory.

Without DMA

NVMe SSD → CPU → Main Memory

This creates high CPU utilization.

With DMA

NVMe SSD ————→ Main Memory

\ DMA /

The CPU does not need to copy each data block.

Step 6: CPU Performs Other Operations

While DMA transfers data:

DMA → Transfers Data

CPU → Performs Other Tasks

Therefore, CPU utilization is reduced.

Step 7: Transfer Completion Interrupt

After the complete data block is transferred, the DMA/NVMe controller generates an interrupt.

Data Transfer Complete

↓

DMA Controller

↓

Interrupt Request

↓

Interrupt Controller

↓

CPU

Step 8: CPU Executes the Completion ISR

The CPU executes a short Interrupt Service Routine.

The ISR:

1. Checks transfer status.
2. Confirms successful completion.
3. Updates the I/O status.
4. Notifies the operating system/application.

Interrupt
↓
Completion ISR
↓
Check Status
↓
Update Status
↓
Return

Data Flow

Control Path

CPU ⟶ NVMe Command ⟶ NVMe Controller

Data Path

NVMe SSD ⟶ PCIe ⟶ DMA ⟶ Main Memory

Completion Path

DMA/NVMe ⟶ Interrupt ⟶ CPU

Why DMA Is Used?

DMA is used because:

- It transfers bulk data directly.
- CPU does not handle every byte.
- CPU utilization is reduced.
- High throughput can be achieved.
- It is suitable for NVMe storage.

Why Interrupt Notification Is Used?

Interrupt notification is better than continuous polling.

Polling

CPU → Check Status
CPU → Check Status
CPU → Check Status

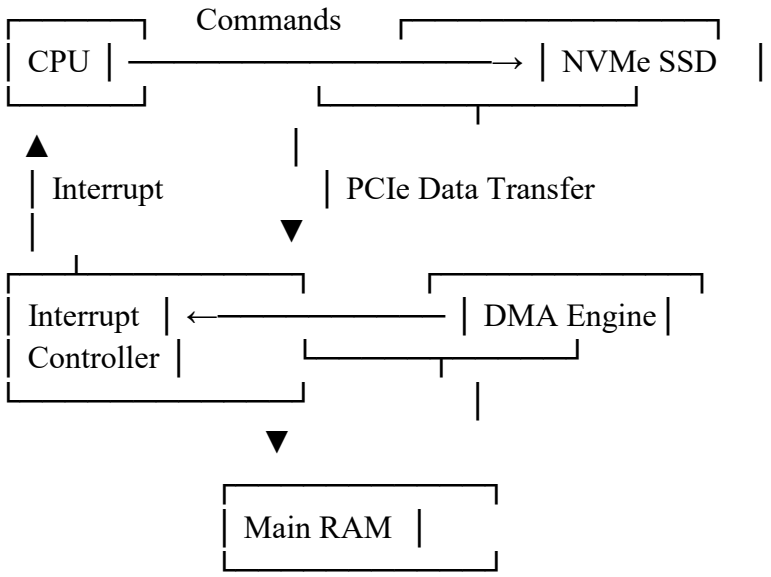
CPU → Check Status
This wastes CPU time.

Interrupt-Based Completion

CPU → Start Transfer
↓
CPU Does Other Work
↓
Transfer Complete
↓
Interrupt
↓
CPU

Thus, interrupts reduce unnecessary CPU activity.

Final Architecture



Result

Thus, the designed I/O subsystem uses NVMe over PCIe for high-speed communication and DMA for direct bulk data transfer between the NVMe SSD and main memory. The CPU only initializes the transfer and handles the completion interrupt. Therefore, the system can achieve high throughput greater than 3 GB/s with minimal CPU utilization.

3. Construct a Multi-Device Communication Architecture

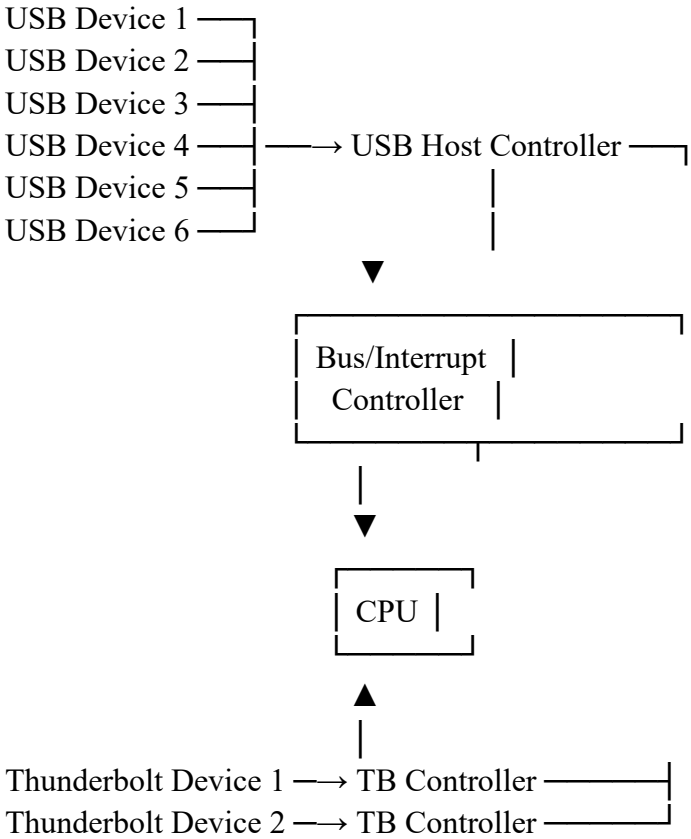
Given Constraints

- **6 peripherals using USB**
- **2 high-speed devices using Thunderbolt**
- **Avoid interrupt starvation**
- **Use hardware and software interrupts**
- **Ensure fair bus arbitration**

Aim

To design a multi-device communication architecture that supports 6 USB peripherals and 2 Thunderbolt devices while ensuring fair bus access and preventing interrupt starvation.

Block Diagram



Components Used

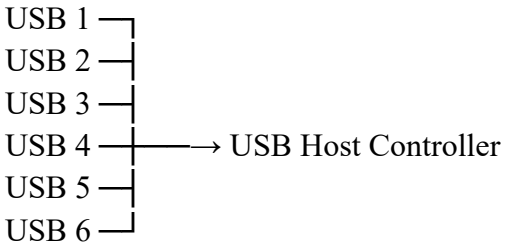
Component	Number	Function
USB peripherals	6	Standard peripheral communication
Thunderbolt devices	2	High-speed data communication
USB Host Controller	1	Controls USB devices
Thunderbolt Controller	1	Controls Thunderbolt devices
Interrupt Controller	1	Manages interrupt requests
CPU	1	Processes device operations

Component	Number	Function
Bus Arbiter	1	Provides fair bus access

Working Procedure

Step 1: Connect the USB Devices

The six peripherals are connected through the USB Host Controller.



The USB controller manages communication between the CPU and USB devices.

Step 2: Connect the Thunderbolt Devices

The two high-speed devices are connected through the Thunderbolt controller.

Thunderbolt Device 1 ⟶ TB Controller

Thunderbolt Device 2 ⟶ TB Controller

Thunderbolt is used for high-speed data transfer.

Step 3: Use Hardware Interrupts

Hardware interrupts are generated for urgent device events such as:

- Data transfer completion
- Device connection
- Device errors
- High-speed Thunderbolt events

Device Event

↓

Hardware Interrupt

↓

Interrupt Controller

↓

CPU

Step 4: Use Software Interrupts

Software interrupts are used for non-critical tasks.

Examples:

- Background data processing
- Device status updates
- Deferred processing
- Logging operations

Hardware ISR

↓

Store Important Information

↓

Generate Software Interrupt
↓
Background Processing
This keeps the hardware ISR short.

Step 5: Interrupt Priority Handling

A priority mechanism is used.

Priority	Device/Event
High	Critical Thunderbolt transfer
Medium	USB transfer completion
Low	Normal peripheral events
Background	Software interrupts

For example:

High Priority
↓
Thunderbolt Critical Event

Medium Priority
↓
USB Data Transfer

Low Priority
↓
Normal Device Event

Lowest Priority
↓
Software Interrupt

Step 6: Prevent Interrupt Starvation

Interrupt starvation happens when low-priority devices never receive CPU service because high-priority interrupts continuously occur.
To avoid this:

1. Keep ISRs Short
Interrupt → Short ISR → Return

2. Use Fair Scheduling
Devices with the same priority are serviced fairly.

3. Use Interrupt Queues
Pending interrupts are stored until they are processed.

4. Use Software Interrupts
Non-critical work is moved outside the hardware ISR.

5. Use Priority Aging
If a device waits for too long, its priority can be increased.

Waiting Time Increases
↓
Priority Increases
↓
Device Gets Service

Step 7: Fair Bus Arbitration

A **round-robin bus arbitration** method is selected.

Example Order

USB 1 → USB 2 → USB 3 → USB 4
↓
USB 6 ← USB 5
↓
TB 1 → TB 2
↓
Repeat

Another representation:

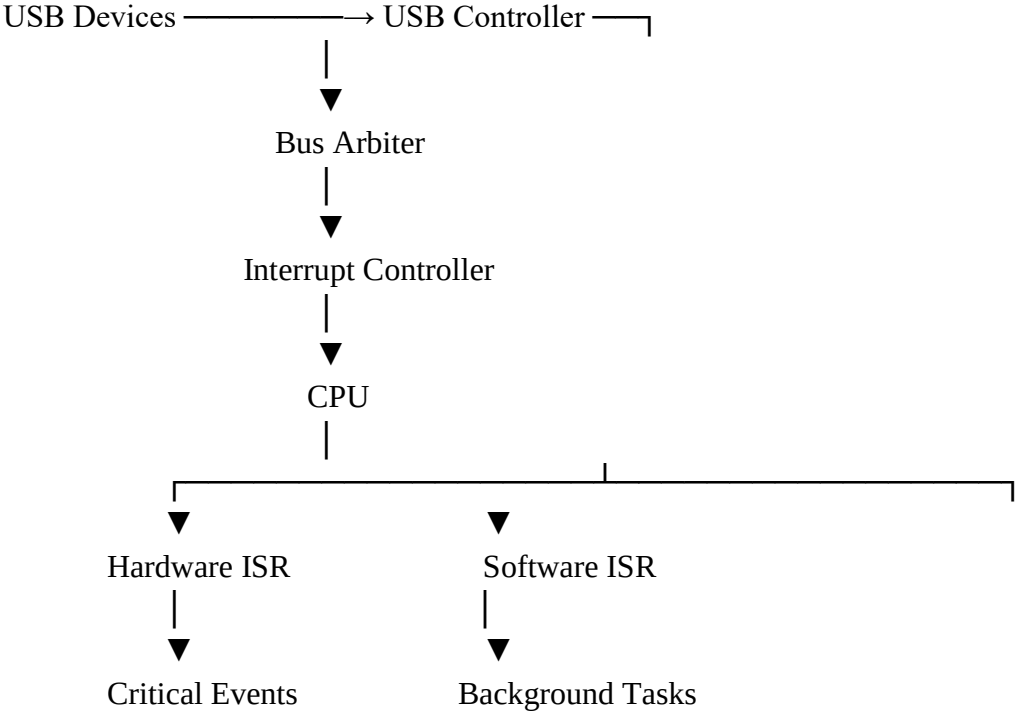
USB1 → USB2 → USB3 → USB4 → USB5 → USB6 → TB1 → TB2
↑ |
└──────────── Repeat ────────────┘

Working of Round-Robin Arbitration

1. A device requests access to the communication bus.
2. The bus arbiter checks the current position.
3. The next waiting device receives access.
4. After completion, the next device is selected.
5. The process repeats continuously.

This prevents one device from occupying the bus continuously.

Complete Data Flow



Thunderbolt Devices → TB Controller ——————————┘

Final Design Decisions

Requirement	Selected Solution
6 peripherals	USB Host Controller
2 high-speed devices	Thunderbolt Controller
Urgent events	Hardware interrupts
Background tasks	Software interrupts
Interrupt starvation	Priority + aging + short ISRs
Fair bus access	Round-robin arbitration

Result

Thus, the proposed multi-device communication architecture supports 6 USB peripherals and 2 Thunderbolt high-speed devices. Hardware interrupts are used for critical device events, while software interrupts handle background tasks. Round-robin bus arbitration ensures fair access, and priority scheduling with aging prevents interrupt starvation. Therefore, the system provides efficient, fair, and reliable multi-device communication.

4. Construct a Data Acquisition System

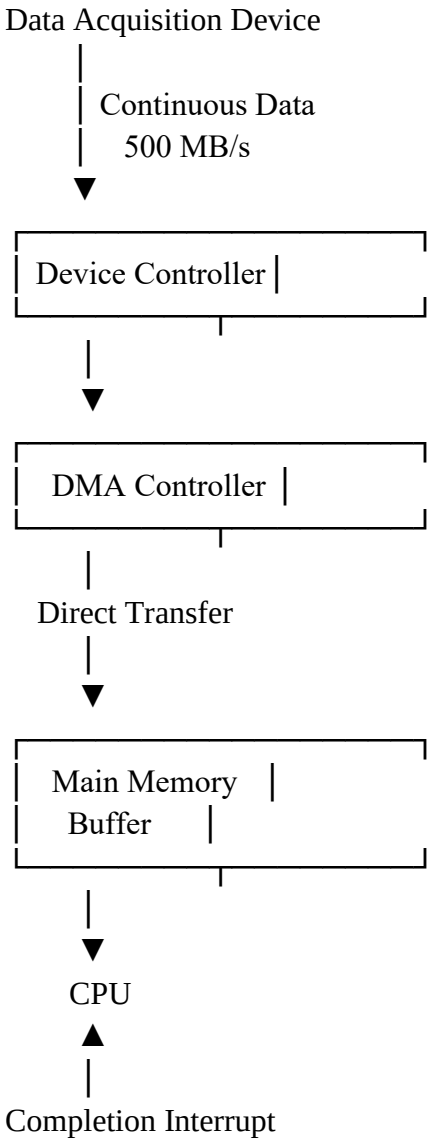
Given Constraints

- Continuous data stream at **500 MB/s**
- Minimal CPU intervention is required
- Compare **DMA transfer** with **interrupt-driven I/O**
- Use **memory-mapped I/O** for device registers

Aim

To design a high-speed data acquisition system capable of continuously receiving data at **500 MB/s** while minimizing CPU involvement.

Block Diagram



Device Registers ——→ Memory-Mapped I/O

Components Used

Component	Function
Data Acquisition Device	Generates continuous data
Device Controller	Controls data acquisition
DMA Controller	Transfers data directly to memory
Main Memory	Stores acquired data
CPU	Configures and processes data
Interrupt Controller	Sends completion notification

Step 1: Configure Device Registers

The device registers are accessed using **memory-mapped I/O**.

CPU
 |
 ▼
Memory Address
 |
 ▼
Device Register

The CPU configures:

- Data acquisition mode
- Sampling rate
- Buffer address
- Transfer size
- Start/stop operation

Step 2: Start Continuous Data Acquisition

The data acquisition device continuously generates data at:

500 MB/s

The system must transfer this large amount of data efficiently.

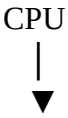
Data Acquisition Device
 |
 | 500 MB/s
 ▼
Device Controller

Method 1: Interrupt-Driven I/O

Working

In interrupt-driven I/O, the device interrupts the CPU whenever data is available.

Device
 |
 ▼
Interrupt
 |
 ▼



Process

1. The device generates data.
2. An interrupt is generated.
3. The CPU stops its current task.
4. The CPU executes the ISR.
5. The CPU reads the data.
6. The CPU stores the data in memory.
7. The CPU returns to its previous task.

Problem

At a continuous speed of **500 MB/s**, this can generate a very large number of interrupts.
Data → Interrupt → CPU
Data → Interrupt → CPU
Data → Interrupt → CPU
Data → Interrupt → CPU
This causes:

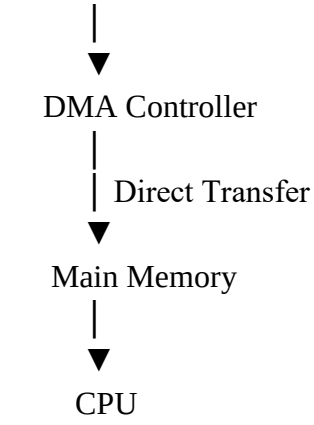
- High CPU utilization
- Frequent context switching
- Increased interrupt overhead
- Possible data loss

Therefore, interrupt-driven I/O is **not the best choice** for this high-speed continuous stream.

Method 2: DMA-Based Transfer

Working

DMA transfers data directly from the device to main memory.

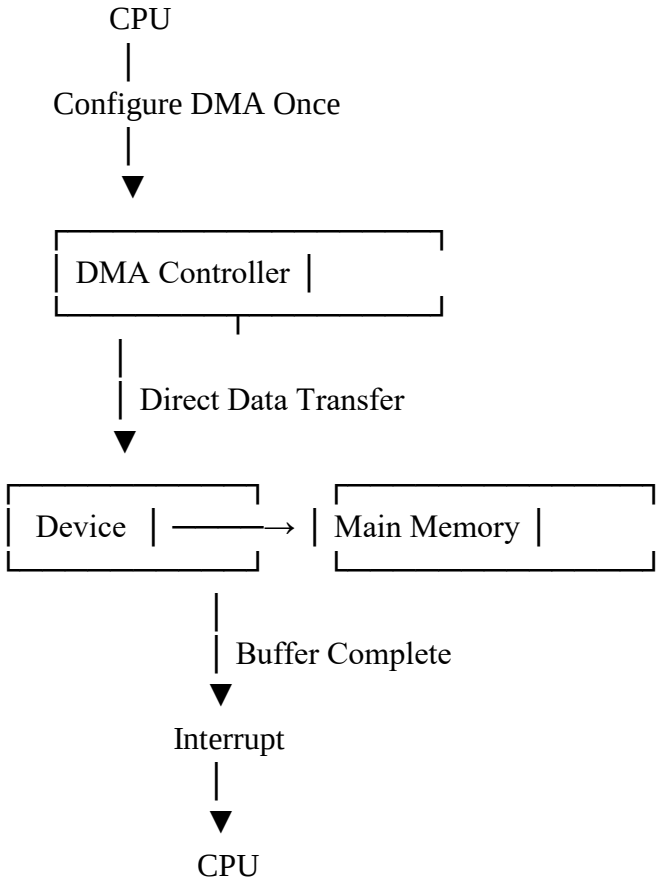


Process

1. The CPU configures the DMA controller.
2. The CPU provides the memory buffer address.

3. The data acquisition device starts generating data.
4. DMA transfers the data directly to main memory.
5. The CPU performs other tasks.
6. When a buffer becomes full, DMA generates an interrupt.
7. The CPU processes the completed buffer.

DMA Data Flow



Comparison: Interrupt-Driven I/O vs DMA

Feature	Interrupt-Driven I/O	DMA Transfer
CPU involvement	High	Very Low
Data transfer	CPU moves data	DMA moves data
Interrupt frequency	Very High	Low
Speed	Limited	High
CPU utilization	High	Low
Suitable for 500 MB/s	No	Yes
Continuous transfer	Difficult	Efficient

Why DMA Is Better?

For a 500 MB/s continuous data stream:

Interrupt-Driven I/O

Device → CPU → Memory

The CPU must repeatedly handle data transfers.

DMA-Based I/O

Device → DMA → Main Memory

↓

Interrupt on

Buffer Completion

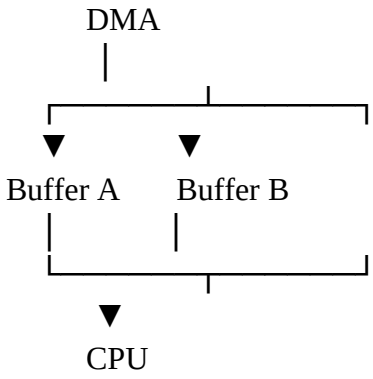
The CPU is only involved during:

- Initial configuration
- Buffer completion
- Error handling

Thus, CPU intervention is minimal.

Use of Buffering

For continuous data acquisition, multiple buffers can be used.



Working

Time 1:

DMA → Buffer A

CPU → Processes Buffer B

Time 2:

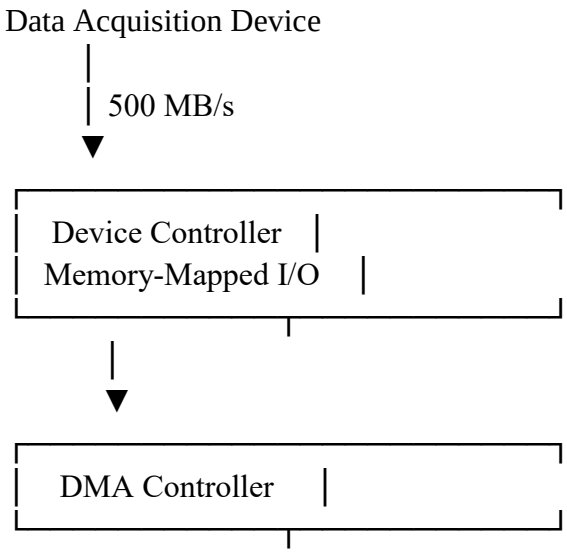
DMA → Buffer B

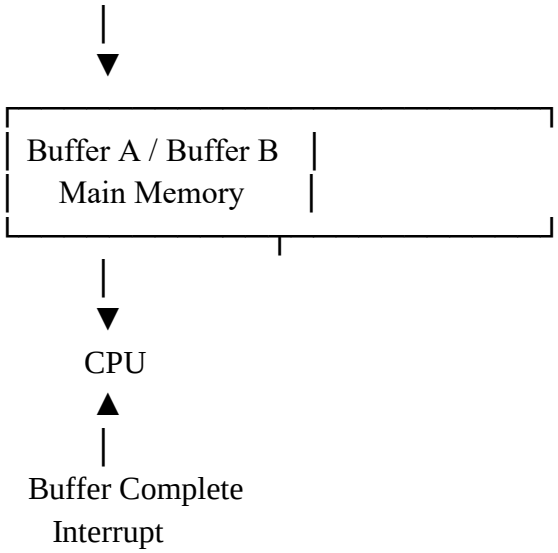
CPU → Processes Buffer A

This is called **double buffering**.

It helps prevent data loss during continuous acquisition.

Final Architecture





Result

Thus, the data acquisition system uses memory-mapped I/O for configuring device registers and DMA for transferring the continuous 500 MB/s data stream directly to main memory. Compared with interrupt-driven I/O, DMA significantly reduces CPU intervention and interrupt overhead. Double buffering ensures continuous data transfer with reduced risk of data loss. Therefore, DMA is the most suitable solution for the given constraints.

5. Construct an Interrupt Handling Mechanism

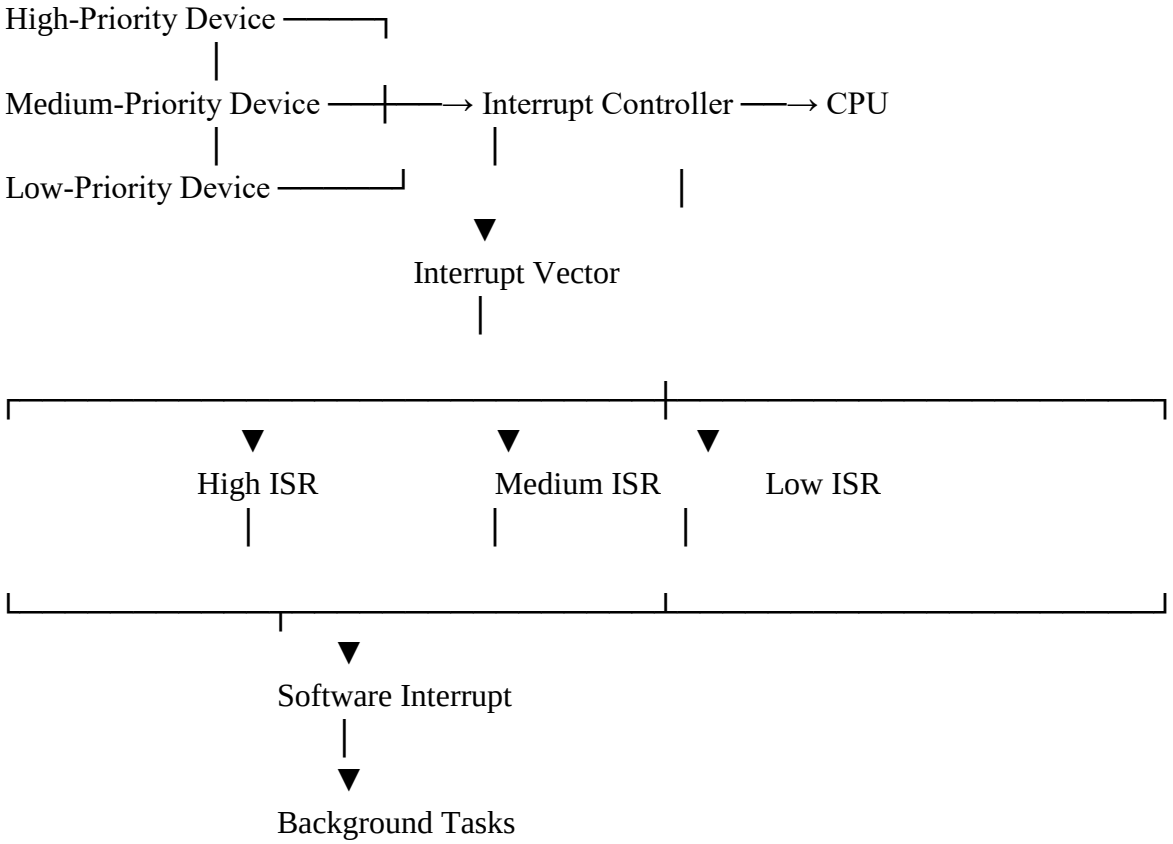
Given Constraints

- Support **nested interrupts**
- Maximum interrupt latency < **10 µs**
- Use **vectored interrupts** for high-priority devices
- Use **software interrupts** for background tasks

Aim

To design an interrupt handling mechanism that supports nested interrupts, provides a response time of less than **10 µs** for critical devices, and separates high-priority hardware events from background tasks.

Block Diagram



Interrupt Priority Levels

Priority Level	Type	Example
Level 1 – Highest	Vectored hardware interrupt	Emergency/Critical device
Level 2	Vectored hardware interrupt	High-speed device
Level 3	Normal hardware interrupt	General peripheral
Level 4 – Lowest	Software interrupt	Background task

Step 1: Device Generates an Interrupt

When a device requires CPU attention, it sends an interrupt request.

Device Event

Interrupt Request

Interrupt Controller

CPU

The interrupt controller identifies the priority of the interrupt.

Step 2: Use Vectored Interrupts for High-Priority Devices

Each high-priority device is assigned a specific interrupt vector.

Device	Vector	ISR
Critical Device	Vector 1	High-Priority ISR
High-Speed Device	Vector 2	Medium-Priority ISR
General Device	Vector 3	Low-Priority ISR

Working

High-Priority Interrupt

↓

Interrupt Vector Number

↓

CPU Directly Finds ISR

↓

Execute ISR

Vectored interrupts reduce latency because the CPU does not need to search for the source of the interrupt.

Step 3: Nested Interrupt Handling

Nested interrupts allow a high-priority interrupt to interrupt a currently running low-priority ISR.

Example

Initially, a low-priority ISR is executing:

CPU

Low-Priority ISR Running

Now, a high-priority interrupt occurs:

High-Priority Interrupt

Save Low-Priority ISR State

Execute High-Priority ISR

After completion:

High-Priority ISR Complete

Restore Previous State

Continue Low-Priority ISR

Nested Interrupt Flow

Low-Priority ISR Running

High-Priority Interrupt Occurs

Save Current CPU Context

Execute High-Priority ISR

Complete High-Priority ISR

Restore CPU Context

Resume Low-Priority ISR

Step 4: Keep Interrupt Latency Below 10 µs

The following methods are used:

1. Use Vectored Interrupts

Interrupt → Vector → Direct ISR

This avoids checking multiple devices.

2. Keep ISR Short

The ISR should only perform essential operations:

Interrupt

↓

Read Status

↓

Save Important Data

↓

Clear Interrupt

↓

Return

Complex processing should not be performed inside the ISR.

3. Use Fixed Priority

High-priority interrupts are serviced immediately.
Priority 1 > Priority 2 > Priority 3

4. Minimize Context Saving

Only essential CPU registers should be saved.

5. Avoid Long Interrupt Masking

Interrupts should not be disabled for a long time.

Step 5: Use Software Interrupts for Background Tasks

Software interrupts are used for non-critical tasks.
Examples:

- Data processing
- Logging
- Updating displays
- Background calculations
- Communication processing

Flow

Hardware Interrupt

Short Hardware ISR

Schedule Software Interrupt

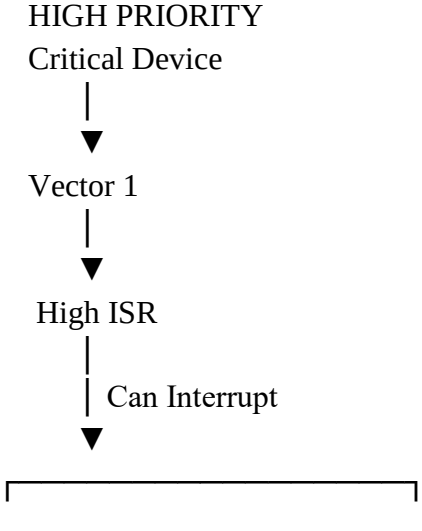
Return Quickly

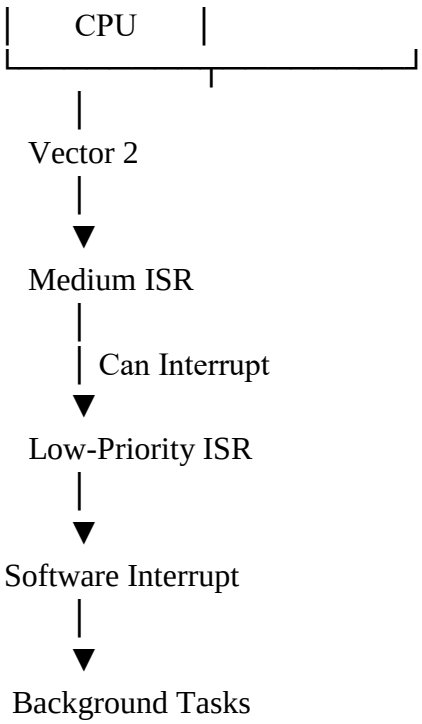
Software Interrupt Executes

Background Processing

This ensures that the hardware ISR remains short and fast.

Complete Interrupt Handling Architecture





Example of Nested Interrupts

Suppose the CPU is executing a low-priority task.

Time 1:

CPU → Low-Priority ISR

A medium-priority interrupt occurs.

Time 2:

Low ISR Paused

↓

Medium ISR Executes

Then a critical interrupt occurs.

Time 3:

Medium ISR Paused

↓

High-Priority ISR Executes

After the critical ISR completes:

High ISR Complete

↓

Resume Medium ISR

↓

Resume Low ISR

Final Design Summary

Requirement	**Selected Solution**
Nested interrupts	Priority-based nesting
Latency < 10 µs	Short ISR + vectored interrupt
High-priority devices	Vectored hardware interrupts
Background tasks	Software interrupts

Requirement	Selected Solution
Fast ISR selection	Interrupt vector table
Resume interrupted task	Context save and restore

Result

Thus, the proposed interrupt handling mechanism uses priority-based nested interrupts to allow high-priority events to interrupt lower-priority processing. Vectored interrupts provide direct access to the required ISR, helping maintain interrupt latency below 10 µs. Software interrupts are used for background tasks, reducing the workload inside hardware ISRs and ensuring efficient real-time system operation.